TALLER DE INTEGRACIÓN II

Crear endpoints para publicar, listar y buscar arriendos

Revisión de implementación, contrato e integración

Proyecto Roomies - ¿Hay cupo? | Código revisado: commit 63b8493

Se revisaron los tres endpoints de la aplicación principal, sus validaciones, consultas SQL, pruebas y consumidores de la página. Esta entrega es una revisión: no se modificó código ni se publicaron o alteraron avisos reales.

OPERACIÓN	RUTA	RESULTADO DEL FLUJO NORMAL
Publicar	POST /api/publicaciones	201 con ID y mensaje; guarda campos e imagen opcional.
Listar	GET /api/publicaciones	200 con un array de avisos abiertos, en orden descendente.
Buscar	GET /api/publicaciones/buscar	200 con el mismo contrato y filtros combinables.

Veredicto

Los tres endpoints existen, están conectados con la página y funcionan en los casos normales probados. No los considero listos para producción: publicar usa una identidad de demostración y los registros antiguos incompletos pueden hacer fallar todo el listado.

Hay además una limitación menor de la interfaz: se puede publicar una habitación, pero el selector de vivienda del inicio no permite filtrar solo habitaciones. El filtro sí existe en la API.

Evidencia

53 pruebas Python del proyecto correctas, incluida cobertura directa de publicar/listar/buscar; suite JavaScript correcta y comprobaciones adicionales con conexiones, archivos y red simulados. Se comprobó la nulabilidad de columnas reales mediante lectura de estructura, sin consultar los avisos.

Estos endpoints están en main/main.py, puerto 8000. El proceso publicaciones_service.main:app del puerto 8001 sigue siendo la preparación independiente que solo expone /health.

02 / REVISIÓN DE ENDPOINTS

Archivos y carpetas que participan

Rutas relativas a la raíz del repositorio. Las referencias corresponden al código revisado y pueden cambiar con futuras ediciones.

Archivo y referencia	Responsabilidad
main/ main.py Línea 199: def obtener_publicaciones	GET de listado y búsqueda; construcción SQL y transformación de la respuesta. POST crear_publicacion está en la misma aplicación.
main/ main.py Línea 120: def preparar_imagen	Validación de archivos. autor_de_prueba obtiene la identidad de demostración usada al publicar.
publicaciones_service/ schemas.py Línea 111: class PublicacionResumen	Contrato de salida del listado/búsqueda y respuesta de creación. Preferencias compartidas de mascotas.
Creacion_De_publicacion/ publicacion.js Línea 96: formDataEnvio.append("titulo"	Mapea los controles HTML al formulario de la API, incluye foto opcional y controla guardado/errores.
Creacion_De_publicacion/ publicacion.html Línea 80: id="form-crear"	Campos, límites y opciones para publicar un aviso.
PaginaPrincipal/ HayCupo.js Línea 188: let ultimaSolicitud	Carga inicial, búsqueda con cuatro filtros, tarjetas y protección frente a respuestas antiguas.
PaginaPrincipal/ hayCupo.html Línea 82: id="form-busqueda"	Formulario de búsqueda; filtros y contenedor de resultados.
tests/ test_publicaciones.py Línea 153: test_crear_guarda_lista	Pruebas HTTP, validación, SQL y casos de error con conexiones simuladas.
tests/ test_publicaciones_ui.cjs Línea 259: async function comprobarInterfaz	Comprobaciones de comportamiento con DOM y fetch simulados.
main/ README.md Línea 63: ## Publicar	Guía actual: campos, filtros, respuestas y ejecución desde la raíz del proyecto.

La ruta histórica /conexiones/publicaciones/buscar pertenece al módulo de conexiones, utiliza otro contrato y no es la que llama esta página. No se debe intercambiar sin adaptar sus nombres, filtros y datos.

03 / REVISIÓN DE ENDPOINTS

Publicar: contrato del POST

POST /api/publicaciones acepta multipart/form-data con imagen opcional o application/x-www-form-urlencoded cuando no hay archivo. El cuerpo JSON no es el formato definido para esta operación.

Campo	Regla de entrada
titulo	Obligatorio, de 1 a 200 caracteres. Se recorta y no puede contener solo espacios.
descripcion	Obligatoria, de 1 a 5000 caracteres; tampoco admite solo espacios.
ubicacion	Obligatoria, de 1 a 250 caracteres; se recorta.
precio	Decimal no negativo, hasta 14 dígitos y 2 decimales. Rechaza NaN e Infinity.
categoria	busco u ofrezco.
tipo / tipo_vivienda	Arrendador o Compartida; casa, departamento o habitacion.
cupos_totales	Entero entre 1 y 2147483647.
mascota	si/no; por defecto no.
tipos_mascotas_aceptadas	Lista JSON enviada como texto. Si acepta mascotas debe indicar al menos un tipo.
condiciones_mascotas	Hasta 2000 caracteres. Obligatorias si se elige otro animal.
permitir_comentarios	Booleano; por defecto false.
imagen	JPG, PNG o WEBP válidos por contenido, máximo 5 MB. No basta con cambiar la extensión.

Guardado y respuesta

Inserta en haycupo.publicacion con estado Disponible y devuelve el ID generado. Cuando hay foto, escribe el archivo y registra su URL en haycupo.imagen_publicacion dentro del flujo de la transacción. Si falla el guardado, intenta retirar el archivo nuevo.

```
{"id": 42, "mensaje": "Publicación creada"}
```

201 indica creación. Datos inválidos responden 422; foto demasiado grande, 413; error de guardado, 500 con un mensaje genérico. El autor no se recibe como un campo del formulario: proviene de la dependencia de identidad actual.

04 / REVISIÓN DE ENDPOINTS

Listar y buscar: filtros y respuesta

Los dos GET comparten obtener_publicaciones y el contrato PublicacionResumen. Sin filtros devuelven el mismo listado. Los filtros se combinan con AND: cada resultado debe cumplir todos los filtros enviados.

Parámetro	Efecto
consulta	Parte de la ubicación, sin distinguir mayúsculas; máximo 250 caracteres. Blanco se ignora. %, _ y la barra se buscan literalmente.
personas	1 y 2 exigen esa cantidad exacta. 3 significa tres o más cupos; no admite 4 como valor del filtro.
tipo	casa, departamento o habitacion; compara tipo_vivienda.
mascotas	si o no; compara mascota.
precio_min / precio_max	Límites inclusivos no negativos. Hasta 14 dígitos y 2 decimales; mínimo no puede superar al máximo.
categoria	busco u ofrezco. Sin filtro incluye ambas categorías.
limite	Entre 1 y 100. Omitirlo devuelve todos los resultados; no existe un límite predeterminado.
offset	Cantidad de resultados a omitir. Desde 0 hasta 9223372036854775807; por defecto 0.

Qué se devuelve

Un array JSON plano. Cada aviso aparece una vez, aunque tenga varias imágenes. Se elige portada por orden e id_imagen y se ordenan los avisos por id_publicacion descendente. Sin coincidencias se devuelve [].


```
[{"id":42, "titulo":"Habitación", "precio":180000.0,
  "ubicacion":"Temuco", "cupos_totales":4,
  "cupos_disponibles":4, "estado":"Disponible",
  "tipo_vivienda":"casa", "imagen":null,
  "categoria":"ofrezco", "descripcion":"Amoblada",
  "mascota":"no", "tipos_mascotas_aceptadas":[],
  "condiciones_mascotas":null}]
```


Se excluyen los avisos Cerrada. Un estado antiguo NULL se interpreta como Disponible. La descripción prefiere contenido cuando tiene texto y usa descripcion como respaldo. Los valores de filtros inválidos responden 422 antes de consultar la base.

05 / REVISIÓN DE ENDPOINTS

Conexión entre la página y los endpoints

Crear publicación

Control del HTML	Campo enviado a la API
titulo / categoria / ubicacion / precio	Se envían con esos mismos nombres.
tipo (vivienda)	tipo_vivienda. No debe confundirse con el tipo Arrendador/Compartida del backend.
contenido	descripcion.
cupos	cupos_totales; cantidad exacta.
Mascotas y condiciones	mascota, tipos_mascotas_aceptadas como JSON y condiciones_mascotas.
Archivo elegido	imagen. Si no se elige, no se envía un archivo vacío.

Al crear, el cliente fija tipo=Compartida y permitir_comentarios=false. La API permite otros valores, pero la pantalla actual no ofrece todos esos controles. Tras respuesta correcta redirige a /inicio; ante errores conserva el formulario y muestra un mensaje.

Listado y búsqueda

```
Abrir /inicio
  -> GET /api/publicaciones -> tarjetas
Consultar o pulsar Enter
  -> GET /api/publicaciones/buscar?... -> resultados filtrados
```

• La UI envía ubicación, personas, vivienda y mascotas mediante URLSearchParams, respetando caracteres especiales.

• Un contador de solicitudes impide que una respuesta antigua reemplace la última búsqueda, incluso cuando la antigua falla.

• Los títulos y ubicaciones se muestran escapados; una búsqueda sin coincidencias muestra el estado vacío. Un error actual limpia los resultados y permite volver a consultar.

Limitación menor de la interfaz

El formulario permite publicar habitacion y la API admite tipo=habitacion, pero el selector del inicio solo ofrece casa y departamento. Falta la opción Habitación para poder pedirla desde esa pantalla. Precio, categoría y paginación también están disponibles en API, pero no tienen controles en la UI actual; no son errores del endpoint.

06 / REVISIÓN DE ENDPOINTS

Hallazgos y recomendaciones concretas

1. El autor se puede suplantar: resolver antes de producción

Ubicación: main/main.py:89 y la dependencia del POST. **Qué es:** autor_de_prueba interpreta directamente la cookie roomies_id. **Problema:** sin cookie publica como usuario 1; con otro número utiliza ese ID sin verificar una sesión. Se reprodujo con una base simulada.

Corrección mínima: obtener el ID de una sesión autenticada y rechazar una publicación sin sesión válida. **Si se omite:** alguien puede crear avisos atribuidos a otro usuario. Validar que el valor sea numérico no resuelve la autenticación.

2. Un aviso incompleto puede hacer fallar todo el listado

Ubicación: main/main.py:267 y PublicacionResumen en schemas.py:111. **Qué es:** se asumen precio, título y cupos válidos para cada fila. **Problema:** precio=NULL provoca error en float(); título o cupos NULL provocan un error al validar la respuesta. Ambos GET devuelven 500 en estas reproducciones.

Corrección mínima: definir la política de datos antiguos, sanear los campos obligatorios y alinear las restricciones de la base con el contrato. Si se necesita tolerancia, debe diseñarse explícitamente; no inventar precios o cupos. **Si se omite:** una fila así puede impedir mostrar también los avisos válidos.

La lectura de estructura confirmó que precio, título, ubicación, cupos, vivienda, categoría y mascota permiten NULL. No se consultaron registros para afirmar que hoy exista un aviso con esos valores. estado=NULL sí está tratado y tiene una prueba de compatibilidad.

3. Completar el selector de vivienda del inicio

Ubicación: PaginaPrincipal/hayCupo.html:103. **Problema:** no se puede filtrar solo habitaciones desde la UI. **Corrección:** añadir la opción value="habitacion". **Si se omite:** habrá que consultar esa categoría mediante la API o buscar sin ese filtro. No bloquea el endpoint.

Nota de crecimiento y documentación

Devolver todo cuando no se envía limite está documentado y no se considera un fallo de la entrega académica. Si crece el volumen, definir paginación predeterminada y conectarla a la página. La guía actual es main/README.md; documentacion_de_api conserva instrucciones antiguas de carpeta que deben unificarse para evitar errores al iniciar.

07 / REVISIÓN DE ENDPOINTS

Pruebas realizadas y límites

53 pruebas Python del proyecto pasaron. No todas pertenecen a estos tres endpoints. La cobertura directamente relacionada se distribuye así:

Cobertura persistente	Cantidad	Qué verifica
Publicar	7	Campos, preferencias, normalización, datos inválidos, imagen inválida y error de base.
Listar y buscar	8	Contrato, portada única, filtros, escape de texto, consulta vacía, límites y errores.
Casos compartidos	2	Imagen con demasiados píxeles y estado antiguo NULL. También cubren otras rutas.
Otras áreas del proyecto	36	Edición, cierre, modelos, validación y configuración fuera del foco principal.

Comprobaciones adicionales correctas

• OpenAPI declara las tres operaciones y sus respuestas; /buscar no se interpreta como un ID.

• PNG válido: POST responde 201 y asocia la imagen al ID creado, sin escribir un archivo real en la prueba.

• Fallo simulado al guardar/confirmar: responde 500 genérico y ejecuta la limpieza del archivo nuevo. Esto no demuestra un rollback real en PostgreSQL.

• Texto con %, _, barra y un intento de inyección permanece en parámetros SQL. No se incorpora como código a la consulta.

• Precio mínimo mayor que máximo responde 422 sin conectar; consulta en blanco no agrega ILIKE.

• Mapeo del formulario y búsqueda con caracteres especiales, respuestas fuera de orden, estado vacío y error: correctos con DOM/fetch simulados.

Casos que revelaron problemas

Las comprobaciones de autor sin sesión y de campos NULL reprodujeron los hallazgos de la página 6. No son pruebas de aceptación superadas: documentan comportamientos pendientes de corregir.

Qué no se comprobó

No hubo creación o modificación de avisos reales, prueba de persistencia completa, recorrido en navegador ni prueba de carga. La estructura de columnas se inspeccionó en modo solo lectura; los escenarios funcionales usaron conexiones simuladas. Las comprobaciones adicionales se ejecutaron en memoria y no se agregaron al repositorio.

08 / REVISIÓN DE ENDPOINTS

Cómo probarlos y conclusión

Iniciar la aplicación correcta desde la raíz

```
.\.venv\Scripts\python.exe -m pip install -r main/requirements.txt
.\.venv\Scripts\python.exe -m uvicorn main.main:app --reload --port 8000
```

Configura DATABASE_URL en el entorno o en .env local. Se requieren el esquema haycupo, las tablas y las columnas de mascotas de la migración existente. No incluyas credenciales en el informe, capturas o ejemplos públicos.

Abre **http://127.0.0.1:8000/docs**, sección Publicaciones, para consultar el contrato; **/crear** para publicar y **/inicio** para listar y buscar. Live Server no proporciona las rutas /api y /static de FastAPI.

Ejemplos de consulta en PowerShell

```
$base = 'http://127.0.0.1:8000'
Invoke-RestMethod "$base/api/publicaciones"
$filtros = 'consulta=Temuco&personas=3&tipo=casa&mascotas=si'
Invoke-RestMethod "$base/api/publicaciones/buscar?$filtros"
```

Recorrido manual pendiente en un entorno de pruebas

Caso	Resultado esperado
Publicar datos válidos y foto	201 con ID; el aviso y la portada aparecen al recargar.
Campos vacíos, precio negativo o imagen inválida	422; no se crea un aviso incompleto. Foto mayor que 5 MB: 413.
Combinar ubicación, vivienda y personas	Solo resultados que cumplen todos los filtros.
Sin coincidencias	200 con [] y mensaje de vacío en la página.
Cerrar un aviso y consultar nuevamente	No aparece en el listado/búsqueda públicos.
Errores y datos antiguos	Comprobar las correcciones de los hallazgos con pruebas de regresión antes de publicar en producción.

Repetir las pruebas locales

```
.\.venv\Scripts\python.exe -B -m unittest discover -s tests -v
node tests/test_publicaciones_ui.cjs
```

Conclusión: la implementación de publicar, listar y buscar está presente y funciona para entradas y registros válidos. Resolver identidad de demostración y compatibilidad de registros incompletos antes de producción. Esta revisión entrega el PDF; no aplica esas correcciones ni reemplaza los informes anteriores.